

# Revision Game Cards — 2.3 Algorithms

Print and cut along the borders. ✂ Loop cards: each player reads the bottom "Who has...?"; whoever holds the matching "I have..." reads it out, then their own clue — the chain visits every card and loops back to the start.

## Loop cards (dominoes) — cut out & deal

|                                                                                                                                                                  |                                                                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div>1 / 15</div><div>I HAVE</div><div>Merge sort</div><div>WHO HAS...</div><div>the worst-case time complexity of binary search??</div></div>              | <div><div>2 / 15</div><div>I HAVE</div><div><math>O(\log n)</math></div><div>WHO HAS...</div><div>the data structure that is LIFO??</div></div>       |
| <div><div>3 / 15</div><div>I HAVE</div><div>Stack</div><div>WHO HAS...</div><div>the traversal that outputs a BST in ascending order??</div></div>               | <div><div>4 / 15</div><div>I HAVE</div><div>In-order traversal</div><div>WHO HAS...</div><div>the average-case complexity of quick sort??</div></div> |
| <div><div>5 / 15</div><div>I HAVE</div><div><math>O(n \log n)</math></div><div>WHO HAS...</div><div>the search that requires the list to be sorted??</div></div> | <div><div>6 / 15</div><div>I HAVE</div><div>Binary search</div><div>WHO HAS...</div><div>the worst-case complexity of quick sort??</div></div>        |

7 / 15

I HAVE

**$O(n^2)$**

---

WHO HAS...

the data structure that is FIFO??

8 / 15

I HAVE

**Queue**

---

WHO HAS...

the path-finding algorithm that uses a heuristic??

9 / 15

I HAVE

**A\* search**

---

WHO HAS...

the complexity of pushing onto a stack??

10 / 15

I HAVE

**$O(1)$**

---

WHO HAS...

the sort that picks a pivot and partitions the list??

11 / 15

I HAVE

**Quick sort**

---

WHO HAS...

the best-case complexity of insertion sort??

12 / 15

I HAVE

**$O(n)$**

---

WHO HAS...

the traversal order Left to Right to Node??

13 / 15

I HAVE

**Post-order traversal**

---

WHO HAS...

the sort that repeatedly swaps adjacent elements??

14 / 15

I HAVE

**Bubble sort**

---

WHO HAS...

the algorithm that finds the shortest path to all nodes with no heuristic??

I HAVE

## Dijkstra's algorithm

---

WHO HAS...

the sort that recursively splits then merges  
sublists??

## Taboo cards — describe the term without the banned words

### Binary search

---

**Don't say:**

- sorted
- half
- middle
- log

### Merge sort

---

**Don't say:**

- divide
- split
- merge
- recursive

### Big O notation

---

**Don't say:**

- complexity
- time
- growth
- worst

### Dijkstra's algorithm

---

**Don't say:**

- shortest
- path
- weighted
- graph

### Bubble sort

---

**Don't say:**

- swap
- adjacent
- pass
- compare

### Stack

---

**Don't say:**

- LIFO
- push
- pop
- last

### In-order traversal

---

**Don't say:**

- left
- node
- right
- ascending

### Heuristic

---

**Don't say:**

- estimate
- guess
- $h(n)$
- distance

## Top Trumps — sorting & searching (play higher/lower)

Tip: for time complexity, "higher" = worse (slower). Compare the agreed stat; best card wins.

### Bubble sort

|              |                            |
|--------------|----------------------------|
| Best         | <b><math>O(n)</math></b>   |
| Average      | <b><math>O(n^2)</math></b> |
| Worst        | <b><math>O(n^2)</math></b> |
| Space        | <b><math>O(1)</math></b>   |
| Needs Sorted | <b>No</b>                  |

### Insertion sort

|              |                            |
|--------------|----------------------------|
| Best         | <b><math>O(n)</math></b>   |
| Average      | <b><math>O(n^2)</math></b> |
| Worst        | <b><math>O(n^2)</math></b> |
| Space        | <b><math>O(1)</math></b>   |
| Needs Sorted | <b>No</b>                  |

### Merge sort

|              |                                 |
|--------------|---------------------------------|
| Best         | <b><math>O(n \log n)</math></b> |
| Average      | <b><math>O(n \log n)</math></b> |
| Worst        | <b><math>O(n \log n)</math></b> |
| Space        | <b><math>O(n)</math></b>        |
| Needs Sorted | <b>No</b>                       |

### Quick sort

|              |                                 |
|--------------|---------------------------------|
| Best         | <b><math>O(n \log n)</math></b> |
| Average      | <b><math>O(n \log n)</math></b> |
| Worst        | <b><math>O(n^2)</math></b>      |
| Space        | <b><math>O(\log n)</math></b>   |
| Needs Sorted | <b>No</b>                       |

### Linear search

|              |                          |
|--------------|--------------------------|
| Best         | <b><math>O(1)</math></b> |
| Average      | <b><math>O(n)</math></b> |
| Worst        | <b><math>O(n)</math></b> |
| Space        | <b><math>O(1)</math></b> |
| Needs Sorted | <b>No</b>                |

### Binary search

|              |                               |
|--------------|-------------------------------|
| Best         | <b><math>O(1)</math></b>      |
| Average      | <b><math>O(\log n)</math></b> |
| Worst        | <b><math>O(\log n)</math></b> |
| Space        | <b><math>O(1)</math></b>      |
| Needs Sorted | <b>Yes</b>                    |

## Quick-fire — clue & answer (self-test / quiz-quiz-trade)

| #  | Clue                                                                | Answer        |
|----|---------------------------------------------------------------------|---------------|
| 1  | The time complexity of binary search (average/worst).               | $O(\log n)$   |
| 2  | The time complexity of a single loop over $n$ items.                | $O(n)$        |
| 3  | The worst-case time complexity of quick sort.                       | $O(n^2)$      |
| 4  | The guaranteed time complexity of merge sort in all cases.          | $O(n \log n)$ |
| 5  | The time complexity of pushing onto a stack.                        | $O(1)$        |
| 6  | The search algorithm that needs the data sorted first.              | Binary search |
| 7  | The sort that picks a pivot and partitions the list.                | Quick sort    |
| 8  | The traversal: Left to Node to Right.                               | In-order      |
| 9  | The path-finding algorithm that adds a heuristic $h(n)$ to $g(n)$ . | A* search     |
| 10 | The data structure that is FIFO.                                    | Queue         |
| 11 | The sort that bubbles the largest value to the end each pass.       | Bubble sort   |
| 12 | The space complexity of merge sort.                                 | $O(n)$        |